K L UNIVERSITY

FRESHMAN ENGINEERING DEPARTMENT

A Project Based Lab Report

On

DYNAMIC TRANSPORT LOGISTICS &

ROUTE OPTIMIZATION TOOL (DTL-RO)

SUBMITTED BY:

ID NUMBER	NAME
2500031866	D.VINAY KUMAR REDDY

UNDER THE ESTEEMED GUIDANCE OF

Dr. N Sreeram

ASSOCIATE PROFESSOR

KL UNIVERSITY

Green fields, Vaddeswaram – 522 502

Guntur Dt., AP, India.

DEPARTMENT OF BASIC ENGINEERING SCIENCES

CERTIFICATE

This is to certify that the project based laboratory report entitled **"Dynamic Transport Logistics & Route Optimization Tool"** submitted by L. Varun Babu, G. Avinash, T. Durga Sai Aditya, . D Vinay Kumar Reddy bearing Regd. No. 2500031781, 2500031771, 2500031853, 2500031866 tothe **Department of COMPUTER SICENCE ENGINEERING ,**
KL University in partial fulfillment of
the requirements for the completion of a project based Laboratory in "Computational Thinking for Problem Solving (24SC1102)" course in I B Tech I Semester, is a Bonafede record of the work carried out by him/her under my supervision during the academic year 2024 – 2025.

PROJECT SUPERVISOR

Dr. N Sreeram

HEAD OF THE DEPARTMENT

Dr. D. HARITHA

ACKNOWLEDGEMENTS

It is great pleasure for me to express my gratitude to our honorable President.

Sri. Koneru Satyanarayana, for giving the opportunity and platform with facilities in accomplishing the project-based laboratory report.

I express sincere gratitude to our Director **Dr. A. Jagdeesh** for his administration towards our academic growth.

I express sincere gratitude to our Coordinator and HOD-BES **Dr. D. Haritha** for her leadership and constant motivation provided in successful completion of our academic semester. I record it as my privilege to deeply thank you for providing us with the efficient faculty and facilities to make our ideas into reality.

I express my sincere thanks to our project supervisor **Dr. N Sreeram** for her novel association of ideas, encouragement, appreciation, and intellectual zeal which motivated us to venture this project successfully.

Finally, we are pleased to acknowledge our indebtedness to all those who devoted themselves directly orindirectly to making this project a success.

Name:

D. Vinay Kumar Reddy

ID No:

2500031866

ABSTRACT

This project is aimed at developing a **Dynamic Transport Logistics and Route Optimization Tool** for efficient management of delivery operations. The system is designed to optimize transportation by assigning orders to vehicles and determining the best possible routes. This project helps in reducing travel time, minimizing fuel consumption, and improving overall delivery efficiency.

The logistics optimization system allows users to manage orders, vehicles, and delivery routes in a structured manner. The system can handle multiple delivery requests and assign them to available vehicles based on capacity and location. Route optimization algorithms are used to ensure that deliveries are completed in the shortest possible path while satisfying constraints such as vehicle capacity.

The objective of this project is to automate logistics planning and reduce manual effort involved in route selection. The system can be accessed at any time and provides flexibility in managing dynamic delivery requests. It eliminates inefficiencies in traditional delivery systems where routes are planned manually.

This system provides complete information about assigned routes, vehicle utilization, and delivery sequence. It also supports dynamic updates where new orders can be added and routes can be re-optimized accordingly. The tool is highly useful for logistics companies, delivery services, and supply chain management.

This project provides a smart and efficient solution for modern transportation problems by improving resource utilization and reducing operational costs.

Key Words: logistics, route optimization, vehicle routing, delivery system, transportation

INDEX

S.NO	TITLE	PAGE NO
1	Introduction	6
2	Aim of the Project	7
2.1	Advantages & Disadvantages	8
2.2	Future Implementation	8
3	Software & Hardware Details	9
4	Data Flow Diagram	10-11
5	Algorithm for each module	12
6	Implementation	13-15
7	Integration and System Testing	16-18
8	Conclusion	18

INTRODUCTION

Welcome to the Dynamic Transport Logistics and Route Optimization Tool, a smart and efficient system designed to improve delivery and transportation management. This system is developed to provide a faster, cleaner, and more optimized solution for handling logistics operations. It helps users manage delivery orders, assign vehicles, and determine the best routes for transportation.

Users can interact with the system to add delivery orders, view available vehicles, and generate optimized routes at any time. The system automatically calculates the most efficient path based on distance and vehicle capacity, reducing manual effort and improving accuracy. It ensures that deliveries are completed in minimum time and cost.

Whenever a new order is added, the system processes the data and assigns it to the most suitable vehicle. The entire route plan is generated dynamically, and users can view the sequence of deliveries along with route details. This system eliminates the need for manual route planning, which is often time-consuming and inefficient.

The tool also supports flexibility by allowing updates in orders and re-optimization of routes. It is highly useful for delivery services, logistics companies, and supply chain management systems where efficient transportation plays a key role.

- **Key Features**
- Easy to understand and use
- Structured and modular design
- Produces efficient route optimization results
- Reduces travel time and operational cost
- Can be implemented across different platforms

AIM

The main purpose of the Dynamic Transport Logistics and Route Optimization Tool is to provide an efficient and automated solution for managing delivery operations. It helps in optimizing transportation by assigning orders to vehicles and determining the best possible routes.

This system automates the process of route planning and order allocation, reducing manual effort and improving accuracy. Once the data is entered into the system, the user can easily manage delivery requests without the need for complex manual calculations. The system also ensures efficient utilization of resources such as vehicles and fuel, thereby reducing operational costs.

One of the key features of the system is its ability to dynamically update routes when new orders are added or existing orders are modified. This improves flexibility and responsiveness in logistics operations.

The modules in this project are:

→ To provide efficient and optimized route planning for deliveries
→ To assign orders to vehicles based on capacity and location
→ To minimize travel time and transportation cost
→ To automate logistics and reduce manual workload

ADVANTAGES

• Reduces travel time and fuel consumption through optimized routing
• Minimizes manual effort in planning delivery routes
• Improves efficiency of logistics and transportation systems
• Ensures better utilization of vehicle capacity
• Provides accurate and reliable route planning
• Can handle multiple orders and vehicles simultaneously

DISADVANTAGES

• Requires computational resources for processing optimization algorithms
• Accuracy depends on the quality of input data (locations, demand, etc.)
• Basic models may not consider real-time traffic conditions
• Initial setup and understanding may be complex for beginners

FUTURE IMPLEMENTATION

• Integration with real-time traffic data for better route optimization
• GPS tracking system for live vehicle monitoring
• Mobile application for drivers and administrators
• AI-based prediction for demand and route planning
• Support for electric vehicles with charging constraints
• Real-time notifications and alerts for delivery updates

SYSTEM REQUIREMENTS

➢ SOFTWARE REQUIREMENTS:

The major software requirements of the project are as follows:
Language: Python

Operating system**:** Windows 10 or more.

Technical requirements: Monitor, CPU, Keyboard, Mouse, etc...

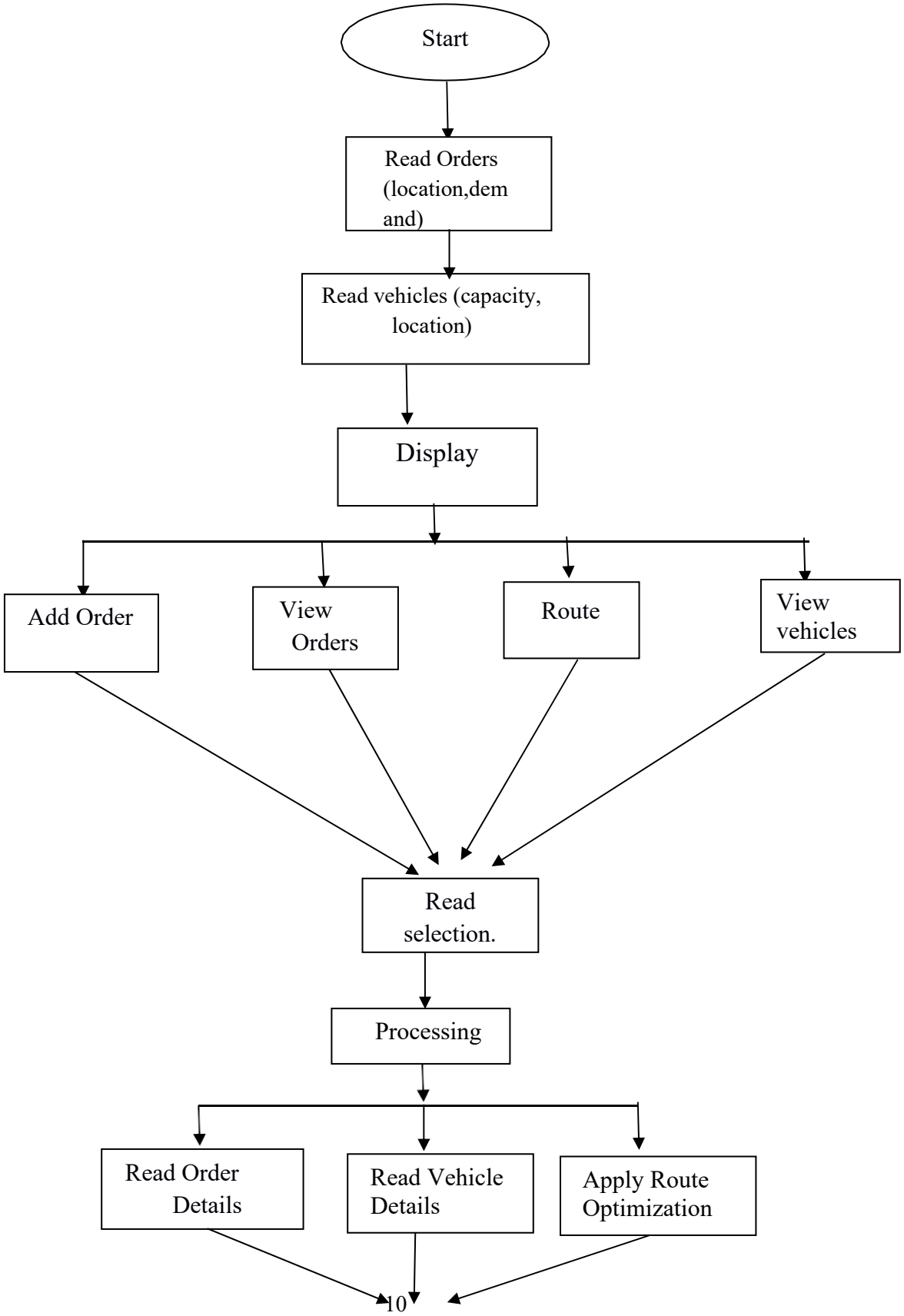
➢ HARDWARE REQUIREMENTS:

The hardware requirements that map towards the software are as follows:

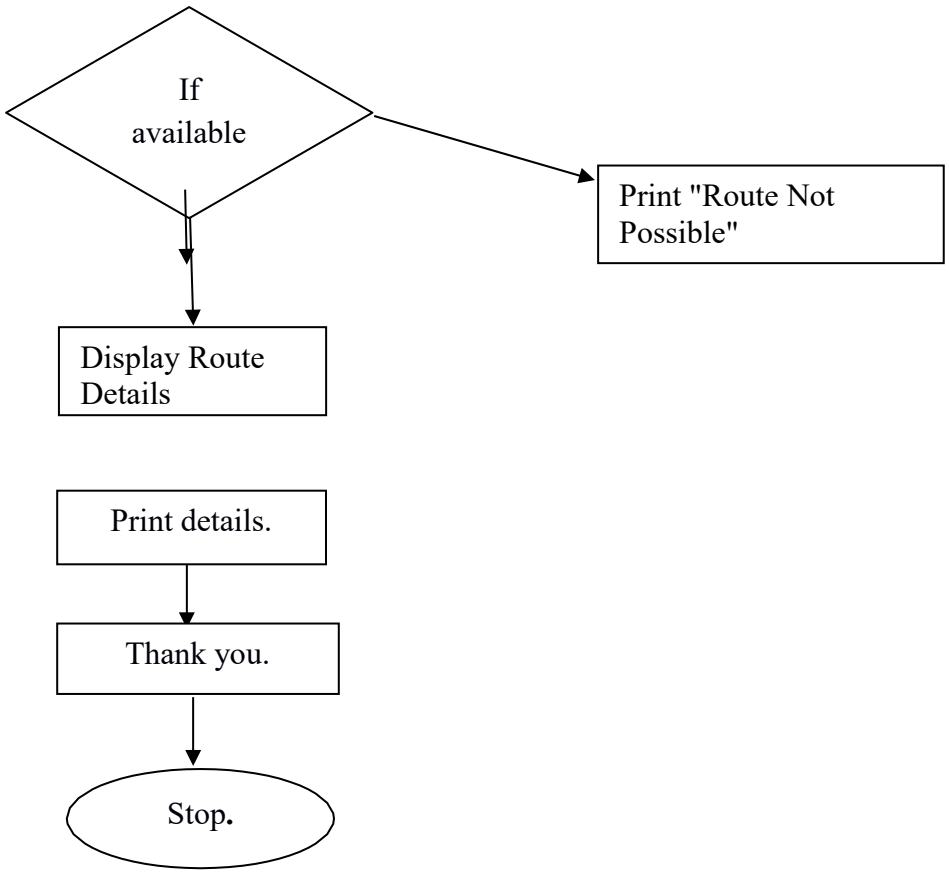
RAM: 8 GB.

Processor: i5/i7.

DATA FLOW DIAGRAM



S


If
available
Print "Route Not
Possible"
Display Route
Details
Print details.
Thank you.
Stop.

ALGORITHM

Step-1: Start

Step-2: Read number of orders

Step-3: For each order
Read order ID, location (x, y), demand

Step-4: Read number of vehicles

Step-5: For each vehicle
Read vehicle ID, capacity, starting location

Step-6: Display menu options
1. Add Order
2 . View Orders
3. Optimize Routes
4. View Vehicles

Step-7: Read user selection

Step-8: If selection = Add Order
Read new order details and store

Step-9: If selection = View Orders
Display all orders

Step-10: If selection = Optimize Routes
Initialize all vehicles at depot
While there are unassigned orders
For each vehicle
Find nearest order
Check capacity constraint
Assign order to vehicle
Update vehicle location
End While

Step-11: Check if all orders are assigned
If yes → Generate routes
Else → Print "Route Not Possible"

Step-12: Display routes for each vehicle

Step-13: Calculate total distance

Step-14: Print route details and cost

Step-15: Stop

IMPLEMENTATION

```
import math

class Order:
    def __init__(self, oid, x, y, demand):
        self.oid = oid
        self.x = x
        self.y = y
        self.demand = demand

class Vehicle:
    def __init__(self, vid, capacity):
        self.vid = vid
        self.capacity = capacity
        self.remaining = capacity
        self.route = []
        self.x = 0
        self.y = 0

def distance(x1, y1, x2, y2):
    return math.sqrt((x1 - x2)**2 + (y1 - y2)**2)

def optimize_routes(orders, vehicles):
    unassigned = orders[:]

    while unassigned:
        progress = False
        for v in vehicles:
            nearest = None
            min_dist = float('inf')

            for o in unassigned:
                if o.demand <= v.remaining:
                    d = distance(v.x, v.y, o.x, o.y)
                    if d < min_dist:
                        min_dist = d
                        nearest = o

            if nearest:
                v.route.append(nearest)
                v.remaining -= nearest.demand
                v.x, v.y = nearest.x, nearest.y
                unassigned.remove(nearest)
                progress = True

        if not progress:
            print("\nSome orders could not be assigned due to capacity constraints.")
            break

    return vehicles

def display_orders(orders):
    if not orders:
```

```
        print("\nNo orders available.")
        return
    print("\nOrders List:")
    for o in orders:
        print(f"ID: {o.oid}, Location: ({o.x},{o.y}), Demand: {o.demand}")

def display_vehicles(vehicles):
    if not vehicles:
        print("\nNo vehicles available.")
        return
    print("\nVehicles List:")
    for v in vehicles:
        print(f"ID: {v.vid}, Capacity: {v.capacity}")

def display_routes(vehicles):
    for v in vehicles:
        print(f"\nVehicle {v.vid} Route:")
        print("Depot -> ", end="")
        for o in v.route:
            print(o.oid, "->", end=" ")
        print("Depot")

def calculate_distance(vehicle):
    total = 0
    x, y = 0, 0
    for o in vehicle.route:
        total += distance(x, y, o.x, o.y)
        x, y = o.x, o.y
    total += distance(x, y, 0, 0)  # return to depot
    return total

def main():
    orders = []
    vehicles = []

    while True:
        print("\n----- LOGISTICS SYSTEM -----")
        print("1. Add Order")
        print("2. Add Vehicle")
        print("3. View Orders")
        print("4. View Vehicles")
        print("5. Optimize Routes")
        print("6. Exit")

        choice = int(input("Enter your choice: "))

        if choice == 1:
            oid = input("Enter Order ID: ")
            x = float(input("Enter X location: "))
            y = float(input("Enter Y location: "))
            demand = float(input("Enter demand: "))
            orders.append(Order(oid, x, y, demand))

        elif choice == 2:
            vid = input("Enter Vehicle ID: ")
            capacity = float(input("Enter capacity: "))
            vehicles.append(Vehicle(vid, capacity))

        elif choice == 3:
            display_orders(orders)
```

```
        elif choice == 4:
            display_vehicles(vehicles)11

        elif choice == 5:
            optimize_routes(orders, vehicles)
            display_routes(vehicles)
            print("\n--- Distance Details ---")
            for v in vehicles:
                print(f"Vehicle {v.vid} Distance: {calculate_distance(v):.2f}")

        elif choice == 6:
            print("Exiting...")
            break

        else:
            print("Invalid choice!")

if __name__ == "__main__":
    main()
```

INTEGRATION AND SYSTEM TESTING OUTPUTS

Screen Shots:

```python
import math

class Order:
    def __init__(self, oid, x, y, demand):
        self.oid = oid
        self.x = x
        self.y = y
        self.demand = demand

class Vehicle:
    def __init__(self, vid, capacity):
        self.vid = vid
        self.capacity = capacity
        self.remaining = capacity
        self.route = []
        self.x = 0
        self.y = 0

def distance(x1, y1, x2, y2):
    return math.sqrt((x1 - x2)**2 + (y1 - y2)**2)

def optimize_routes(orders, vehicles):
    unassigned = orders[:]

    while unassigned:
        progress = False
```

```
----- LOGISTICS SYSTEM -----
1. Add Order
2. Add Vehicle
3. View Orders
4. View Vehicles
5. Optimize Routes
6. Exit
Enter your choice: 2
Enter Vehicle ID:
```

main.py

```python
import math

class Order:
    def __init__(self, oid, x, y, demand):
        self.oid = oid
        self.x = x
        self.y = y
        self.demand = demand

class Vehicle:
    def __init__(self, vid, capacity):
        self.vid = vid
        self.capacity = capacity
        self.remaining = capacity
        self.route = []
        self.x = 0
        self.y = 0

def distance(x1, y1, x2, y2):
    return math.sqrt((x1 - x2)**2 + (y1 - y2)**2)

def optimize_routes(orders, vehicles):
    unassigned = orders[:]

    while unassigned:
        progress = False
```

```
----- LOGISTICS SYSTEM -----
1. Add Order
2. Add Vehicle
3. View Orders
4. View Vehicles
5. Optimize Routes
6. Exit
Enter your choice: 2
Enter Vehicle ID: AP 16 VL 9999
Enter capacity:
```

main.py

```python
import math

class Order:
    def __init__(self, oid, x, y, demand):
        self.oid = oid
        self.x = x
        self.y = y
        self.demand = demand

class Vehicle:
    def __init__(self, vid, capacity):
        self.vid = vid
        self.capacity = capacity
        self.remaining = capacity
        self.route = []
        self.x = 0
        self.y = 0

def distance(x1, y1, x2, y2):
    return math.sqrt((x1 - x2)**2 + (y1 - y2)**2)

def optimize_routes(orders, vehicles):
    unassigned = orders[:]

    while unassigned:
        progress = False
```

Output

```
----- LOGISTICS SYSTEM -----
1. Add Order
2. Add Vehicle
3. View Orders
4. View Vehicles
5. Optimize Routes
6. Exit
Enter your choice: 2
Enter Vehicle ID: AP 16 VL 9999
Enter capacity: 50

----- LOGISTICS SYSTEM -----
1. Add Order
2. Add Vehicle
3. View Orders
4. View Vehicles
5. Optimize Routes
6. Exit
Enter your choice:
```

main.py

```python
import math

class Order:
    def __init__(self, oid, x, y, demand):
        self.oid = oid
        self.x = x
        self.y = y
        self.demand = demand

class Vehicle:
    def __init__(self, vid, capacity):
        self.vid = vid
        self.capacity = capacity
        self.remaining = capacity
        self.route = []
        self.x = 0
        self.y = 0

def distance(x1, y1, x2, y2):
    return math.sqrt((x1 - x2)**2 + (y1 - y2)**2)

def optimize_routes(orders, vehicles):
    unassigned = orders[:]

    while unassigned:
        progress = False
```

Output

```
----- LOGISTICS SYSTEM -----
1. Add Order
2. Add Vehicle
3. View Orders
4. View Vehicles
5. Optimize Routes
6. Exit
Enter your choice: 2
Enter Vehicle ID: AP 16 VL 9999
Enter capacity: 50

----- LOGISTICS SYSTEM -----
1. Add Order
2. Add Vehicle
3. View Orders
4. View Vehicles
5. Optimize Routes
6. Exit
Enter your choice: 6
Exiting...

=== Code Execution Successful ===
```

CONCLUSION

This project has been developed successfully and the performance of the system has been found satisfactory. Use of this interface helps customers in having immediate information about running movies and reserve their seat without wasting their precious time. User friendly Interface also for the admin to add and delete movie information.